

REST ASSURED

API Testing Documentation

A Complete Guide for QA Testers

POJO Classes · Builder Pattern · BDD Given / When / Then

Version 1.0 | Groovy | Gradle | REST Assured 5.x

For Beginners and Junior QA Engineers

1. Introduction

Welcome to the REST Assured API Testing Guide — Groovy Edition! This document teaches you how to write professional, reusable API tests using Groovy with Gradle as the build tool. Groovy is simpler and less verbose than Java, making it ideal for QA automation.

1.1 Why Groovy?

- Less boilerplate — no semicolons, no 'public', shorter syntax
- Native GString interpolation — embed variables directly in strings with \${variable}
- Built-in map and list literals — easier test data creation
- Fully compatible with the Java ecosystem — all REST Assured and Jackson features work

1.2 What You Will Learn

- How to set up a Groovy REST Assured project with Gradle
- How to write POJO classes in Groovy using @JsonProperty
- How to use the Builder Pattern in Groovy with method chaining
- How to create a reusable BaseTest class and extend it in test classes
- How to write BDD-style tests using Given / When / Then
- How to test complex APIs with nested lists and objects

1.3 Project Architecture

Layer	Description
Base Class	Common config: base URL, headers, auth — inherited by all tests
POJO Classes	Groovy classes mirroring the JSON structure of your API
Builder Classes	Fluent builders to construct POJO objects cleanly
Test Classes	Extend BaseTest; contain Given/When/Then test methods
Config File	config.properties stores environment-specific values

2. Project Setup

2.1 build.gradle (Groovy DSL)

This replaces pom.xml. Paste this into your build.gradle file at the root of your project.

```
// build.gradle

plugins {
    id 'groovy' // Enables Groovy compilation
}

group = 'com.yourcompany'
version = '1.0.0'

repositories {
    mavenCentral() // Download dependencies from Maven Central
}

dependencies {

    // Groovy SDK – required for .groovy files to compile
    implementation 'org.apache.groovy:groovy-all:4.0.15'

    // REST Assured – core API testing library
    testImplementation 'io.rest-assured:rest-assured:5.3.2'

    // Jackson – converts Groovy/Java objects <-> JSON automatically
    implementation 'com.fasterxml.jackson.core:jackson-databind:2.15.2'

    // TestNG – test runner framework
    testImplementation 'org.testng:testng:7.8.0'

}

test {
    useTestNG() // Tell Gradle to use TestNG (not JUnit)
}
```

- 📌 Groovy uses single quotes for plain strings: 'hello'
- 📌 Use double quotes for GStrings (string interpolation): "Hello \${name}"
- 📌 No semicolons required at end of lines — Groovy is optional about them.

2.2 settings.gradle

Create this file at the root of your project alongside build.gradle.

```
// settings.gradle
rootProject.name = 'library-api-tests'
```

2.3 Folder Structure

Groovy source files go under `src/main/groovy` and `src/test/groovy` — same idea as Java but .groovy files.

```
src/
├── main/
│   ├── groovy/com/yourcompany/
│   │   ├── config/
│   │   │   └── ConfigManager.groovy    <- Reads config.properties
│   │   ├── pojo/
│   │   │   ├── Library.groovy         <- Top-level POJO
│   │   │   ├── BookList.groovy        <- List element POJO
│   │   │   ├── Author.groovy          <- Nested object POJO
│   │   │   └── Address.groovy         <- Nested object POJO
│   │   └── builder/
│   │       ├── LibraryBuilder.groovy   <- Builder for Library
│   │       ├── BookListBuilder.groovy  <- Builder for BookList
│   │       ├── AuthorBuilder.groovy    <- Builder for Author
│   │       └── AddressBuilder.groovy    <- Builder for Address
│   └── resources/
│       └── config.properties
└── test/
    ├── groovy/com/yourcompany/
    │   ├── base/
    │   │   └── BaseTest.groovy         <- Base class (all tests extend this)
    │   └── tests/
    │       ├── GetLibraryTest.groovy   <- GET tests
    │       ├── CreateLibraryTest.groovy <- POST tests
    │       └── UpdateLibraryTest.groovy <- PUT/PATCH/DELETE tests
```

2.4 config.properties

Store environment-specific values here. Switch environments by changing this one file.

```
# src/main/resources/config.properties
```

```
base.url=https://api.library.example.com
api.version=/v1
api.key=your-api-key-here
timeout=30000
```

2.5 ConfigManager.groovy

Groovy's syntax is cleaner than Java for this utility class — no try-with-resources verbosity.

```
package com.yourcompany.config

/**
 * ConfigManager - loads config.properties once, provides static get() method.
 *
 * Usage anywhere in your code:
 *   ConfigManager.get('base.url')    // returns the value as a String
 */
class ConfigManager {

    // Groovy static initializer - loads file once when class is first used
    private static final Properties props = new Properties()

    static {
        def stream =
            ConfigManager.classLoader.getResourceAsStream('config.properties')
        props.load(stream)
    }

    static String get(String key) {
        return props.getProperty(key)
    }
}
```

2.6 Gradle Commands

Command	What It Does
<code>./gradlew test</code>	Run all tests
<code>./gradlew test --info</code>	Run tests with verbose logging
<code>./gradlew test --tests "GetLibraryTest"</code>	Run only one test class
<code>./gradlew dependencies</code>	Show all resolved dependencies

Command	What It Does
<code>./gradlew clean test</code>	Clean build cache and run fresh

3. POJO Classes in Groovy

3.1 What is a POJO in Groovy?

In Groovy we still call them POJOs (or sometimes POGOs — Plain Old Groovy Objects). The structure is identical to Java POJOs: each class field maps to a JSON field via `@JsonProperty`. The key difference is Groovy syntax is shorter and cleaner.

- ⚡ Rule: One Groovy class per JSON object level. Start from the deepest nested object first.
- 📌 Always add `@JsonIgnoreProperties(ignoreUnknown = true)` — this prevents test failures when the API adds new fields.
- 🔧 Always add a no-argument constructor: `Author() {}` — Jackson needs this to deserialize JSON.

3.2 Example API JSON Structure

This is the complex Library API we will model. Study this JSON before writing any class.

```
{
  "libraryId": 101,
  "libraryName": "City Central Library",
  "isActive": true,
  "contactEmail": "info@library.com",
  "address": {
    "street": "123 Main St",
    "city": "Dhaka",
    "zipCode": "1207"
  },
  "bookList": [
    {
      "bookName": "Clean Code",
      "bookCount": 5,
      "author": {
        "authorName": "Robert C. Martin",
        "nationality": "American"
      },
      "tags": ["programming", "best-practices"]
    }
  ]
}
```

3.3 Author.groovy (Deepest Nested POJO)

Start with the innermost object. Author sits inside BookList, which sits inside Library.

```
package com.yourcompany.pojo

import com.fasterxml.jackson.annotation.JsonIgnoreProperties
import com.fasterxml.jackson.annotation.JsonProperty

/**
 * Author POJO – maps to the 'author' object inside each book.
 *
 * JSON: { "authorName": "Robert C. Martin", "nationality": "American" }
 *
 * Groovy note: No 'public' keyword needed – it is public by default.
 * Groovy note: No semicolons needed at end of lines.
 */
@JsonIgnoreProperties(ignoreUnknown = true)
class Author {

    @JsonProperty('authorName')
    String authorName

    @JsonProperty('nationality')
    String nationality

    // No-argument constructor – required by Jackson for JSON deserialization
    Author() {}
}
```

3.4 BookList.groovy (Mid-Level POJO with List + Nested Object)

BookList is one element of the bookList array. It contains a nested Author object and a List of Strings.

```
package com.yourcompany.pojo

import com.fasterxml.jackson.annotation.JsonIgnoreProperties
import com.fasterxml.jackson.annotation.JsonProperty

/**
 * BookList POJO – maps to each element in the 'bookList' JSON array.
 *
 * Contains:
 *   - Primitive fields (String, int)
 *   - A nested object (Author)
 *   - A list of strings (List<String>)
 */
```

```
@JsonIgnoreProperties(ignoreUnknown = true)
class BookList {

    @JsonProperty('bookName')
    String bookName

    @JsonProperty('bookCount')
    int bookCount

    // Nested Author POJO - Jackson maps the 'author' JSON object here
    @JsonProperty('author')
    Author author

    // List of plain strings - maps to 'tags' JSON array
    @JsonProperty('tags')
    List<String> tags

    BookList() {}
}
```

3.5 Address.groovy

```
package com.yourcompany.pojo

import com.fasterxml.jackson.annotation.JsonIgnoreProperties
import com.fasterxml.jackson.annotation.JsonProperty

@JsonIgnoreProperties(ignoreUnknown = true)
class Address {

    @JsonProperty('street')
    String street

    @JsonProperty('city')
    String city

    @JsonProperty('zipCode')
    String zipCode

    Address() {}
}
```

3.6 Library.groovy (Top-Level / Root POJO)

This is the root object — it holds all nested POJOs. Write this last, after all inner classes exist.

```
package com.yourcompany.pojo

import com.fasterxml.jackson.annotation.JsonIgnoreProperties
import com.fasterxml.jackson.annotation.JsonProperty

/**
 * Library POJO – the ROOT object of the full API response.
 *
 * Contains:
 *   - Primitives : int, String, boolean
 *   - Nested obj : Address
 *   - List       : List<BookList>
 */
@JsonIgnoreProperties(ignoreUnknown = true)
class Library {

    @JsonProperty('libraryId')
    int libraryId

    @JsonProperty('libraryName')
    String libraryName

    @JsonProperty('isActive')
    boolean isActive

    @JsonProperty('contactEmail')
    String contactEmail

    // Nested Address object
    @JsonProperty('address')
    Address address

    // List of BookList POJOs – maps to the JSON 'bookList' array
    @JsonProperty('bookList')
    List<BookList> bookList

    Library() {}
}
```

- ✓ POJO Checklist (Groovy):
- ✓ check Each class has `@JsonIgnoreProperties(ignoreUnknown = true)`

```
F check Every field has @JsonProperty('exactJsonFieldName') — use single quotes in Groovy
C check Every class has ClassName() {} no-arg constructor
F
T check No 'public' keyword needed — Groovy fields are public by default
A check No semicolons — Groovy does not require them
N
T
```

4. Builder Pattern in Groovy

4.1 Why Builder Pattern?

The Builder Pattern creates objects using readable method chaining. In Groovy this is even cleaner than Java because Groovy supports named constructor arguments natively — but explicit builders improve readability in test code and allow partial object construction.

Without Builder	With Builder
<pre>def lib = new Library()lib.libraryName = 'X'lib.isActive = true// 8 more lines...</pre>	

4.2 AuthorBuilder.groovy

```
package com.yourcompany.builder

import com.yourcompany.pojo.Author

/**
 * AuthorBuilder - builds an Author POJO using method chaining.
 *
 * Usage:
 *   def author = new AuthorBuilder()
 *       .authorName('Robert C. Martin')
 *       .nationality('American')
 *       .build()
 *
 * Groovy note: 'def' means infer the type automatically.
 * Groovy note: 'return this' enables chaining - each method returns the builder.
 */
class AuthorBuilder {

    private Author author = new Author()

    AuthorBuilder authorName(String authorName) {
        author.authorName = authorName
        return this // <- returning 'this' enables method chaining
    }

    AuthorBuilder nationality(String nationality) {
        author.nationality = nationality
        return this
    }
}
```

```
    Author build() {
        return author          // <- finalize and return the completed object
    }
}
```

4.3 BookListBuilder.groovy

```
package com.yourcompany.builder

import com.yourcompany.pojo.Author
import com.yourcompany.pojo.BookList

/**
 * BookListBuilder - builds a BookList (one book entry).
 *
 * Usage:
 *   def book = new BookListBuilder()
 *       .bookName('Clean Code')
 *       .bookCount(5)
 *       .author(authorObject)
 *       .tags('programming', 'best-practices')
 *       .build()
 */
class BookListBuilder {

    private BookList book = new BookList()

    BookListBuilder bookName(String bookName) {
        book.bookName = bookName
        return this
    }

    BookListBuilder bookCount(int bookCount) {
        book.bookCount = bookCount
        return this
    }

    BookListBuilder author(Author author) {
        book.author = author
        return this
    }

    // Groovy varargs: .tags('a', 'b', 'c') - collected into a List automatically
    BookListBuilder tags(String... tags) {
```

```
        book.tags = tags.toList()
        return this
    }

    BookList build() {
        return book
    }
}
```

4.4 AddressBuilder.groovy

```
package com.yourcompany.builder

import com.yourcompany.pojo.Address

class AddressBuilder {

    private Address address = new Address()

    AddressBuilder street(String street) {
        address.street = street
        return this
    }

    AddressBuilder city(String city) {
        address.city = city
        return this
    }

    AddressBuilder zipCode(String zipCode) {
        address.zipCode = zipCode
        return this
    }

    Address build() {
        return address
    }
}
```

4.5 LibraryBuilder.groovy (Root Builder)

```
package com.yourcompany.builder
```

```
import com.yourcompany.pojo.Address
import com.yourcompany.pojo.BookList
import com.yourcompany.pojo.Library

/**
 * LibraryBuilder - builds the top-level Library request/response object.
 *
 * Full chained usage:
 *
 * def author = new AuthorBuilder().authorName('Andy
Hunt').nationality('American').build()
 * def book = new
BookListBuilder().bookName('Pragmatic').bookCount(3).author(author).build()
 * def address = new AddressBuilder().street('123
Main').city('Dhaka').zipCode('1207').build()
 *
 * def library = new LibraryBuilder()
 *     .libraryName('City Central Library')
 *     .isActive(true)
 *     .contactEmail('info@library.com')
 *     .address(address)
 *     .bookList(book)      // pass one or more BookList objects
 *     .build()
 */
class LibraryBuilder {

    private Library library = new Library()

    LibraryBuilder libraryName(String libraryName) {
        library.libraryName = libraryName
        return this
    }

    LibraryBuilder isActive(boolean isActive) {
        library.isActive = isActive
        return this
    }

    LibraryBuilder contactEmail(String contactEmail) {
        library.contactEmail = contactEmail
        return this
    }

    LibraryBuilder address(Address address) {
        library.address = address
        return this
    }
}
```

```
// Varargs - accepts one or more BookList objects: .bookList(book1, book2)
LibraryBuilder bookList(BookList... books) {
    library.bookList = books.toList()
    return this
}

Library build() {
    return library
}
}
```

5. Base Class (Reusability Foundation)

5.1 Why a Base Class?

BaseTest.groovy contains everything that every test needs — the base URL, headers, logging, and authentication. All test classes extend BaseTest so they inherit all of this without repeating it.

TIP Golden rule: If you copy-paste the same code into two test classes, it belongs in BaseTest instead. In Groovy, 'extends BaseTest' works exactly the same as in Java.

5.2 BaseTest.groovy

```
package com.yourcompany.base

import com.yourcompany.config.ConfigManager
import io.restassured.RestAssured
import io.restassured.builder.RequestSpecBuilder
import io.restassured.builder.ResponseSpecBuilder
import io.restassured.filter.log.RequestLoggingFilter
import io.restassured.filter.log.ResponseLoggingFilter
import io.restassured.http.ContentType
import io.restassured.specification.RequestSpecification
import io.restassured.specification.ResponseSpecification
import org.testng.annotations.BeforeClass

/**
 * BaseTest – the parent class that ALL test classes extend.
 *
 * What it sets up:
 * 1. Base URL from config.properties
 * 2. Common headers (Content-Type, API Key)
 * 3. Request and response logging filters
 * 4. A shared ResponseSpecification (content-type assertion)
 *
 * How to use in a test class:
 * class GetLibraryTest extends BaseTest { ... }
 */
class BaseTest {

    // 'protected' means child test classes can access these directly
    protected RequestSpecification requestSpec
    protected ResponseSpecification responseSpec
}
```

```
@BeforeClass // Runs once before any test in this class
void setUp() {

    // Build a reusable request spec - applies to every test automatically
    requestSpec = new RequestSpecBuilder()
        .setBaseUrl(ConfigManager.get('base.url')) // e.g.
https://api.library.com
        .setBasePath(ConfigManager.get('api.version')) // e.g. /v1
        .setContentType(ContentType.JSON) // All requests send
JSON
        .addHeader('x-api-key', ConfigManager.get('api.key')) // Auth header
        .addFilter(new RequestLoggingFilter()) // Logs every request
        .addFilter(new ResponseLoggingFilter()) // Logs every
response
        .build()

    // Build a reusable response spec - asserts content-type on every response
    responseSpec = new ResponseSpecBuilder()
        .expectContentType(ContentType.JSON)
        .build()

    // Extra: log full request+response only when a test fails
    RestAssured.enableLoggingOfRequestAndResponseIfValidationFails()
}
}
```

6.1 BDD Given / When / Then — Quick Reference

6.2 GetLibraryTest.groovy

```
        .statusCode(200) // Assert HTTP 200 OK
        .log().all() // Print response (helpful for
debugging)
    }

// — Test 2: Deserialize response into POJO and assert fields —
@Test
void getLibraryById_shouldReturnCorrectName() {

    // Groovy: 'def' infers type - Library library = ... also works
    def library = given()
        .spec(requestSpec)
        .pathParam('id', 101)
    .when()
        .get('/libraries/{id}')
    .then()
        .statusCode(200)
        .extract().as(Library) // <- Deserialize JSON response into Library
    POJO

    // GString interpolation in assertion messages: use double quotes + ${var}
    Assert.assertEquals(library.libraryName, 'City Central Library',
        "Library name mismatch! Got: ${library.libraryName}")
    Assert.assertTrue(library.isActive, 'Library should be active')
    Assert.assertNotNull(library.address, 'Address should not be null')
    Assert.assertEquals(library.address.city, 'Dhaka')
}

// — Test 3: Assert nested list and nested object inside list —
@Test
void getLibraryById_shouldContainBooks() {

    def library = given()
        .spec(requestSpec)
        .pathParam('id', 101)
    .when()
        .get('/libraries/{id}')
    .then()
        .statusCode(200)
        .extract().as(Library)

    // Assert the list
    Assert.assertNotNull(library.bookList, 'Book list should not be null')
    Assert.assertFalse(library.bookList.isEmpty(), 'Book list should not be
empty')

    // Access list elements with Groovy's .get(0) or [0] shorthand
```

```
Assert.assertEquals(library.bookList[0].bookName, 'Clean Code')
Assert.assertEquals(library.bookList[0].bookCount, 5)

// Assert nested object inside the list
Assert.assertEquals(library.bookList[0].author.authorName, 'Robert C.
Martin')
Assert.assertEquals(library.bookList[0].author.nationality, 'American')

// Assert nested list (tags) inside the list element
Assert.assertTrue(library.bookList[0].tags.contains('programming'))
}

// — Test 4: Negative test – resource not found —————
@Test
void getLibraryById_withInvalidId_shouldReturn404() {

    given()
        .spec(requestSpec)
        .pathParam('id', 99999)    // Non-existent ID
    .when()
        .get('/libraries/{id}')
    .then()
        .statusCode(404)           // Assert 404 Not Found
}
}
```

6.3 CreateLibraryTest.groovy (POST with Builder Pattern)

```
package com.yourcompany.tests

import com.yourcompany.base.BaseTest
import com.yourcompany.builder.AddressBuilder
import com.yourcompany.builder.AuthorBuilder
import com.yourcompany.builder.BookListBuilder
import com.yourcompany.builder.LibraryBuilder
import com.yourcompany.pojo.Library
import org.testng.Assert
import org.testng.annotations.Test

import static io.restassured.RestAssured.given

class CreateLibraryTest extends BaseTest {

    // — Private helper: builds a complete test Library —————
    // Reuse this in multiple tests without duplicating builder code
```

```
private Library buildTestLibrary() {

    def author = new AuthorBuilder()
        .authorName('Robert C. Martin')
        .nationality('American')
        .build()

    def book = new BookListBuilder()
        .bookName('Clean Code')
        .bookCount(5)
        .author(author)
        .tags('programming', 'best-practices')
        .build()

    def address = new AddressBuilder()
        .street('123 Main St')
        .city('Dhaka')
        .zipCode('1207')
        .build()

    return new LibraryBuilder()
        .libraryName('City Central Library')
        .isActive(true)
        .contactEmail('info@library.com')
        .address(address)
        .bookList(book)
        .build()
}

// — Test 1: POST creates library — expect 201 —————
@Test
void createLibrary_shouldReturn201() {

    def requestBody = buildTestLibrary()

    given()                                // GIVEN
        .spec(requestSpec)                // Reuse headers from BaseTest
        .body(requestBody)                // Attach Library POJO — REST
Assured serializes to JSON

    .when()                                // WHEN
        .post('/libraries')                // Send POST request

    .then()                                // THEN
        .spec(responseSpec)                // Verify JSON content-type
        .statusCode(201)                    // 201 Created
}
```

```
        .log().all()
    }

    // ——— Test 2: POST response contains created data ———
    @Test
    void createLibrary_responseShouldEchoCreatedData() {

        def requestBody = buildTestLibrary()

        def response = given()
            .spec(requestSpec)
            .body(requestBody)
            .when()
                .post('/libraries')
            .then()
                .statusCode(201)
                .extract().as(Library)

        Assert.assertEquals(response.libraryName, 'City Central Library')
        Assert.assertTrue(response.isActive)
        Assert.assertNotNull(response.libraryId,
            'API should have assigned a new libraryId')

        // Verify nested book data in response
        Assert.assertEquals(response.bookList.size(), 1)
        Assert.assertEquals(response.bookList[0].bookName, 'Clean Code')
        Assert.assertEquals(response.bookList[0].author.authorName, 'Robert C.
Martin')
    }

    // ——— Test 3: POST without required field - expect 400 ———
    @Test
    void createLibrary_missingName_shouldReturn400() {

        def badRequest = new LibraryBuilder()
            .isActive(true)
            .contactEmail('test@library.com')
            // deliberately omit .libraryName() to trigger validation error
            .build()

        given()
            .spec(requestSpec)
            .body(badRequest)
            .when()
                .post('/libraries')
            .then()
                .statusCode(400) // 400 Bad Request
    }
```

```
}  
}
```

6.4 UpdateLibraryTest.groovy (PUT / PATCH / DELETE)

```
package com.yourcompany.tests  
  
import com.yourcompany.base.BaseTest  
import com.yourcompany.builder.LibraryBuilder  
import com.yourcompany.pojo.Library  
import org.testng.Assert  
import org.testng.annotations.Test  
  
import static io.restassured.RestAssured.given  
  
class UpdateLibraryTest extends BaseTest {  
  
    // — Test 1: PUT replaces the full library object —————  
    @Test  
    void updateLibrary_shouldReturn200() {  
  
        def updatedLibrary = new LibraryBuilder()  
            .libraryName('Updated Library Name')  
            .isActive(false)  
            .contactEmail('updated@library.com')  
            .build()  
  
        given()                                // GIVEN  
            .spec(requestSpec)  
            .pathParam('id', 101)  
            .body(updatedLibrary)  
  
        .when()                                // WHEN  
            .put('/libraries/{id}')  
        .then()                                // THEN  
            .statusCode(200)  
            .log().all()  
    }  
  
    // — Test 2: PATCH updates a single field —————  
    @Test  
    void patchLibraryEmail_shouldReturn200() {
```

```
// Groovy Map literal – a clean alternative to a raw JSON string
def patchBody = [contactEmail: 'new@library.com']

def response = given()
    .spec(requestSpec)
    .pathParam('id', 101)
    .body(patchBody) // REST Assured serializes Map to JSON automatically
.when()
    .patch('/libraries/{id}')
.then()
    .statusCode(200)
    .extract().as(Library)

Assert.assertEquals(response.contactEmail, 'new@library.com',
    "Email was not updated! Got: ${response.contactEmail}")
}

// — Test 3: DELETE removes the library —————
@Test
void deleteLibrary_shouldReturn204() {

    given()
        .spec(requestSpec)
        .pathParam('id', 101)
    .when()
        .delete('/libraries/{id}')
    .then()
        .statusCode(204) // 204 No Content = deletion successful
}
}
```

7. Groovy Tips & Quick Reference

7.1 Groovy vs Java: Key Differences in Test Code

Feature	Java	Groovy
Strings	'hello' = compile error"hello" = String	Both work. Use single quotes by default.
Interpolation	"Hi " + name	"Hi \${name}" — cleaner!
List literal	Arrays.asList("a","b")	['a', 'b'] — shorter!
Map literal	new HashMap<>()....	[key: 'value'] — great for PATCH body
List access	list.get(0)	list[0] — same result
Type decl.	Library lib = ...	def lib = ... — type inferred
Public keyword	public class / public void	Not needed — public by default
Semicolons	Required	Optional — omit them
POJO extract	.extract().as(Library.class)	.extract().as(Library) — no .class needed

7.2 HTTP Status Code Cheat Sheet

Code	Name	When to Expect It
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST that created a new resource
204	No Content	Successful DELETE — no response body
400	Bad Request	Missing or invalid fields in request body
401	Unauthorized	Missing or incorrect API key / token
403	Forbidden	Authenticated but not allowed to access this
404	Not Found	Resource with that ID does not exist
500	Server Error	Backend crashed — report to the developer team

7.3 Common Assertions in REST Assured (Groovy)

```
// — Status codes —  
.then().statusCode(200)  
.then().statusCode(201)  
  
// — Body values using JSONPath —  
.then().body('libraryName', equalTo('City Central Library'))
```

```

.then().body('address.city', equalTo('Dhaka'))
.then().body('bookList[0].bookName', equalTo('Clean Code'))
.then().body('bookList[0].author.authorName', equalTo('Robert C. Martin'))
.then().body('bookList.size()', equalTo(2))
.then().body('bookList[0].tags', hasItem('programming'))

// — Headers —————
.then().header('Content-Type', containsString('application/json'))

// — Extract values from response —————
def id    = response.jsonPath().getInt('libraryId')
def name  = response.jsonPath().getString('libraryName')
def tags  = response.jsonPath().getList('bookList[0].tags')

// — Deserialize full response into POJO —————
def library = response.as(Library)           // Groovy – no .class needed
def books   = response.as(BookList[]).toList() // For array responses

// — Groovy map as PATCH body —————
def patchBody = [contactEmail: 'new@email.com', isActive: false]
given().body(patchBody)... // REST Assured auto-serializes to JSON

```

7.4 JSONPath Syntax for Nested Responses

JSONPath Expression	What It Retrieves
libraryName	Top-level string field
address.city	Field inside nested object
bookList[0].bookName	First element of list
bookList[-1].bookName	Last element of list
bookList[0].author.authorName	Nested object inside list item
bookList[0].tags[0]	First item in nested string list
bookList.size()	Total number of books in list
bookList.bookName	All book names collected as a List

7.5 Common Mistakes and Fixes

Mistake	Fix
Jackson cannot deserialize response	Add <code>@JsonIgnoreProperties(ignoreUnknown=true)</code> and <code>ClassName()</code> {} no-arg constructor

Mistake	Fix
@JsonProperty value does not match	Compare EXACTLY with JSON key — case-sensitive. Use single quotes in Groovy.
NullPointerException on nested field	Check if the parent object (e.g. address) is null before accessing child fields
401 Unauthorized in all tests	Check api.key value in config.properties and header name 'x-api-key' in BaseTest
Test pass locally, fail in CI pipeline	Use ConfigManager — never hardcode URLs or keys in test code
Gradle test task not found	Make sure useTestNG() is in build.gradle test block
Groovy files not compiled	Apply 'groovy' plugin in build.gradle and check src/main/groovy folder path

8. Full Flow Summary

Here is how all the pieces connect from configuration to a passing test:

```
1. config.properties      -> Stores base URL and API key
2. ConfigManager          -> Loads config.properties, provides get('key') method
3. BaseTest.setUp()       -> Reads config, builds requestSpec with headers + logging
4. Test class             -> extends BaseTest -> inherits requestSpec automatically
5. Builder classes        -> Build Author -> BookList -> Address -> Library POJOs
6. given()                -> Attaches requestSpec + body (Library POJO is
auto-serialized to JSON)
7. when()                 -> Sends HTTP request (GET / POST / PUT / DELETE)
8. then()                 -> Asserts status code, deserializes JSON -> Library POJO
9. Assert.*               -> Verifies individual field values on the POJO
```

Gradle command to run all tests:

```
./gradlew test
```

Gradle command to run one specific test class:

```
./gradlew test --tests 'com.yourcompany.tests.GetLibraryTest'
```

- ✓ You are ready to write Groovy REST Assured tests! Summary of key rules:
1. One Groovy POJO per JSON object level — start from deepest nested
 2. One Builder per POJO — use method chaining, always return 'this'
 3. BaseTest holds all common config — extend it in every test class
 4. Always use Given / When / Then for readable, maintainable tests
 5. Use def and Groovy map literals [key: value] to keep test code concise
 6. Run tests with ./gradlew test — view reports in build/reports/tests/